

Interakcja w Systemie CAD: Selekcja, Manipulacja i Kamera

W profesjonalnym systemie CAD, interakcja użytkownika z modelem 3D odbywa się na trzech głównych poziomach: wskazywania elementów (Picking), bezpośredniej manipulacji (Direct Manipulation) oraz kontroli widoku (Camera Control). Poniżej znajduje się opis implementacji tych mechanizmów.

1. Wykrywanie Elementów (Picking / Selection)

Moduł **Selection Manager** odpowiada za identyfikację elementu (punktu, krawędzi, ściany), na który wskazuje kursor. Implementacja tego mechanizmu zazwyczaj odbywa się w warstwie pośredniej między UI a Kernelem.

Metoda	Opis	Zastosowanie
Ray Casting	Wysłanie promienia z pozycji kursora w głąb sceny. Testuje się kolizję promienia z obiektami B-Rep.	Najdokładniejsza metoda dla krawędzi i wierzchołków.
Color Picking	Renderowanie sceny do ukrytego bufora, gdzie każdy element ma unikalny kolor (ID). Odczyt koloru pod kursorem daje ID elementu.	Bardzo szybka metoda dla dużych złożań i całych ścian.
BVH (Bounding Volume Hierarchy)	Drzewo otoczek obiektów. Służy do szybkiego odrzucania obiektów, które na pewno nie są pod kursorem.	Optymalizacja wydajności przy tysiącach elementów.

Gdzie to zaimplementować?

Mechanizm ten powinien znajdować się w dedykowanym **Selection Managerze**, który współpracuje z **Wizualizacją** (pobieranie macierzy widoku) oraz **Kernelem** (pobieranie dokładnych danych geometrycznych do testów kolizji).

2. Przesuwanie i Manipulacja Elementami

W systemach parametrycznych (jak CATIA), "przesuwanie" elementu rzadko jest prostą zmianą współrzędnych. Zazwyczaj jest to zmiana parametru w drzewie operacji.

1. **Gizma / Manipulatory:** To specjalne obiekty 3D (strzałki, pierścienie) renderowane nad modelem. Gdy użytkownik "chwyci" strzałkę, system przechwytuje ruch myszy.
2. **Delta Translation:** Ruch myszy jest tłumaczony na wartość liczbową (np. +10mm w osi X).
3. **Update Parametru:** Ta wartość jest wysyłana do **Feature Managera**, który aktualizuje parametr w odpowiedniej cechy (np. Pad.1.Offset).
4. **Re-compute:** Całe drzewo operacji (lub jego fragment) jest przeliczane przez Kernel, co generuje nową geometrię.

3. Obracanie Widoku (Camera Control)

Kontrola kamery jest niezależna od logiki modelu i powinna być zaimplementowana w **Camera Controllerze** wewnątrz warstwy wizualizacji.

- **Algorytm Arcball (Trackball):** Mapuje ruch myszy 2D na powierzchnię wirtualnej kuli otaczającej model. Pozwala to na intuicyjne obracanie widoku w dowolnym kierunku.
- **Kwaterniony (Quaternions):** Do przechowywania orientacji kamery należy używać kwaternionów zamiast kątów Eulera, aby uniknąć problemu "Gimbal Lock" (blokady osi) i zapewnić płynne przejścia.
- **View Matrix:** Wynikowa rotacja i pozycja kamery są zamieniane na macierz widoku (View Matrix), która jest przekazywana do shaderów GPU przy każdym odświeżeniu klatki.

4. Schemat Wywołań Interakcji

mermaid

❗ Failed to render mermaid:

```
Parse error on line 7:
... as Geometric Kernel      Note over U, U
-----^
Expecting 'SOLID_OPEN_ARROW', 'DOTTED_OPEN_ARROW', 'SOLID_ARROW',
'BIDIRECTIONAL_SOLID_ARROW', 'DOTTED_ARROW', 'BIDIRECTIONAL_DOTTED_ARROW',
'SOLID_CROSS', 'DOTTED_CROSS', 'SOLID_POINT', 'DOTTED_POINT', got 'NEWLINE'
```

 Ask Manus to fix

